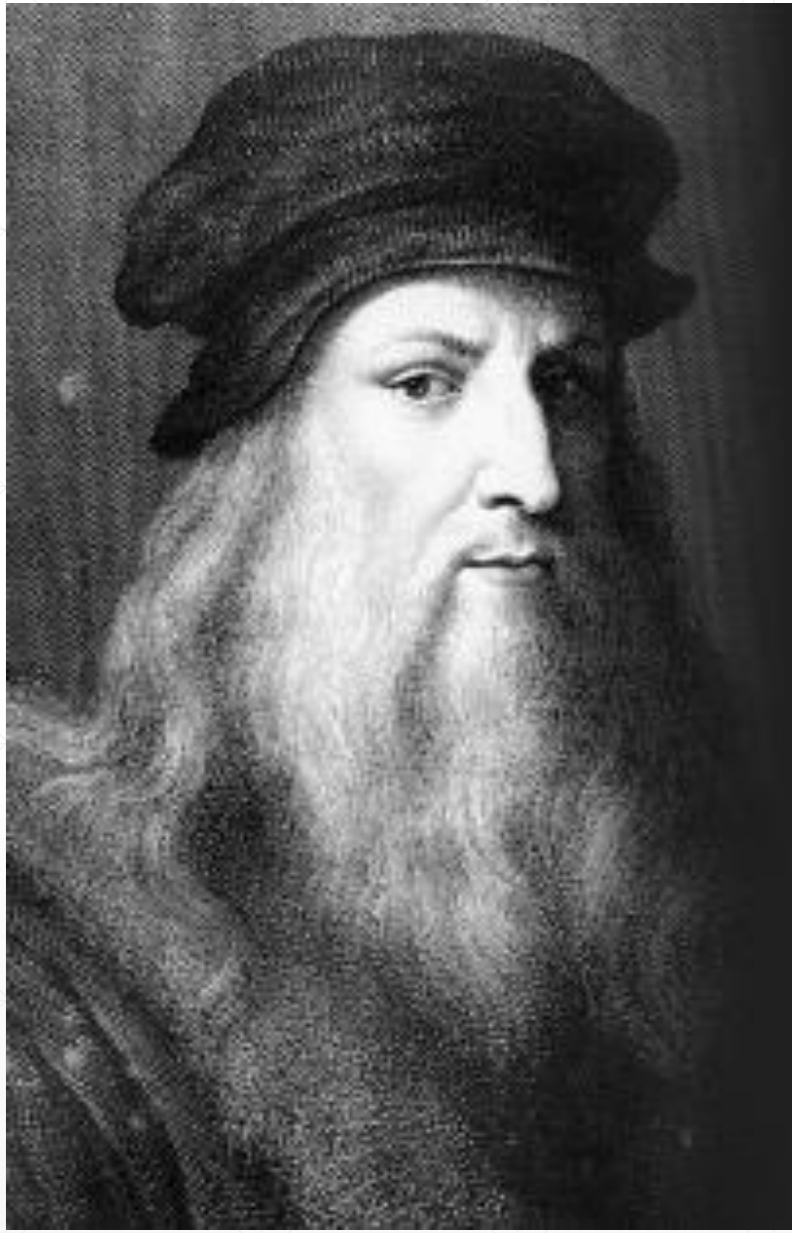
ELE2 – LPG3

Eletiva 2 – Linguagem de Programação 3 (Java)
Prof. Davi Reis

Aula 02 – Orientação a Objetos – Revisão Básica

Prof. Davi dos Reis

https://linktr.ee/prof.davi.reis



Aprender é a única coisa de que a mente nunca se cansa, nunca tem medo e nunca se arrepende.

Leonardo da Vinci

PENSADOR

Conceitos

- Classe
  - Objeto
  - Instância
- Construtor e Destrutor
- Encapsulamento (membros)
  - Atributos
  - Métodos
- Herança
- Polimorfismo

Classe

- Abstrai um conjunto de objetos/entidades com características e comportamentos similares entre si.
- Define (ou descreve) a estrutura e o comportamento que os objetos compartilharão.
- Para isso, dispõe de:
  - Atributos: características ou estados possíveis destes objetos.
  - Métodos: ações que o objeto pode realizar.

Estrutura de um Classe

- Atributos
- Métodos
- Construtores
- Destrutor

- Sintaxe:

```
class NomeDaClasse{
    modificador tipo atributo
    modificador tipo método (tipo parâmetro1, tipo parâmetro2..., tipo parâmetroN){
        //Código...
    }
}
```

Atributos

- São declarações das classes que representam características de instância que possam assumir estados.
- No exemplo, **dia**, **mes** e **ano** são atributos do tipo inteiro da classe **Data**, enquanto **hora**, **minuto** e **segundo** são atributos do tipo inteiro da classe **Horario**. Na classe Principal, nenhum atributo foi declarado.

```java
package aularevisao;

class Data{
    int dia, mes, ano;
}
class Horario{
    int hora, minuto, segundo;
}
public class Principal {
    public static void main(String[] args) {
        Data data = new Data();
        data.ano = 2025; data.mes = 12; data.dia = 31;

        Horario horario = new Horario();
        horario.hora = 23; horario.minuto = 59; horario.segundo = 59;

        System.out.println(String.format("Data: %d/%d/%d",
                data.dia, data.mes, data.ano));

        System.out.println(String.format("Hora: %d:%d:%d",
                horario.hora, horario.minuto, horario.segundo));
    }
}
```

Saída - AulaRevisao (run)

```
run:
Data: 31/12/2025
Hora: 23:59:59
CONSTRUÍDO COM SUCESSO (tempo total: 0 segundos)
```

Métodos

- São declarações das classes que representam ações que a mesma pode executar em função dos valores de seus atributos.
- No exemplo, um método chamado **retornarInfo()** em cada classe retorna uma String representando a data e a hora em função dos valores dos atributos de suas respectivas instâncias.

```java
package aularevisao;

class Data{
    int dia, mes, ano;
    String retornarInfo(){
        return String.format("Data: %d/%d/%d", dia, mes, ano);
    }
}
class Horario{
    int hora, minuto, segundo;
    String retornarInfo(){
        return String.format("Hora: %d:%d:%d", hora, minuto, segundo);
    }
}
public class Principal {
    public static void main(String[] args) {
        Data data = new Data();
        data.ano = 2025; data.mes = 12; data.dia = 31;

        Horario horario = new Horario();
        horario.hora = 23; horario.minuto = 59; horario.segundo = 59;

        System.out.println(data.retornarInfo());
        System.out.println(horario.retornarInfo());
    }
}
```

Saída - AulaRevisao (run)

```
run:
Data: 31/12/2025
Hora: 23:59:59
CONSTRUÍDO COM SUCESSO (tempo total: 0 segundos)
```

Objetos e Instâncias

- Objeto é uma entidade que pode ser física, conceitual ou de software. Ou seja, uma representação genérica.

- Instância é usada com o sentido de exemplo. É a concretização da classe. Ou seja, são os objetos de fato criados e ocupando espaço na memória.

Classe Cachorro:
“Cachorro.txt”

1 Objeto
Cachorro

3 objetos
Cachorro

Instâncias
da classe
Cachorro

Construtor

- Método especial. É disparado automaticamente na instanciação da classe.
- Utilizado basicamente para inicializar os estados dos atributos.
- Deve possuir sempre o mesmo nome da classe.
- Pode ser do tipo **default** ou **não-default.**
- Retorna sempre a instância da classe.
- Normalmente **public**, mas ter outros modificadores de acesso.
- No exemplo a seguir, foram criados dois construtores. Cada um deles é chamado de acordo com a criação da instância, pela lista de argumentos.

Construtor
Exemplo:

```java
package aularevisao;

class Data{
    int dia, mes, ano;
    public Data() {
        dia = mes = ano = 0;
    }
    public Data(int dia, int mes, int ano) {
        this.dia = dia; this.mes = mes; this.ano = ano;
    }
    String retornarInfo(){
        return String.format("Data: %d/%d/%d", dia, mes, ano);
    }
}

public class Principal {
    public static void main(String[] args) {
        Data data = new Data();
        data.ano = 2025; data.mes = 12; data.dia = 31;
        System.out.println(data.retornarInfo());

        Data data2 = new Data(31,12,2029);
        System.out.println(data2.retornarInfo());
    }
}
```

Saída - AulaRevisao (run)

```
Data: 31/12/2025
Data: 31/12/2029
```

Destrutor

- O método Destrutor não pode ser sobrecarregado porque não aceita parâmetros e é executado quando o objeto é excluído.

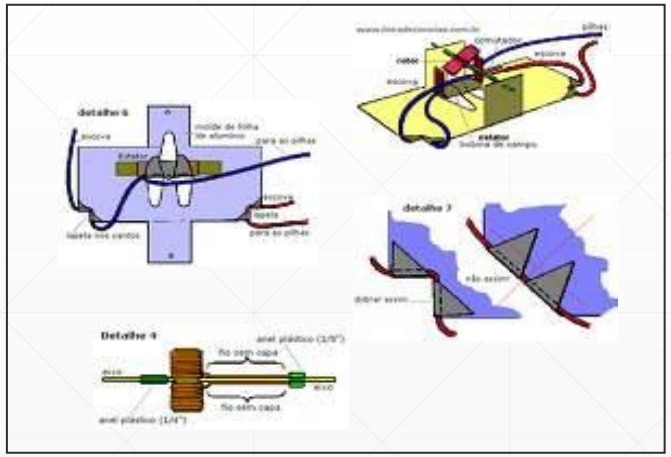
```
obj.finalize();
```

- Não é recomendado para a liberação de recursos, pois o momento em que ele é chamado pelo *garbage collector* pode ser imprevisível.
- Para liberação de recursos, é aconselhável utilizar como “try-finally” ou implementar interfaces como “AutoCloseable”, se necessário.

Encapsulamento

- Mecanismo que permite separar detalhes de funcionamento – <u>características (atributos)</u> e <u>funções (métodos)</u> – de sua interface.
- Para isso, limita o acesso aos membros da classe
  - Exemplo: Para utilizarmos um liquidificador, não precisamos saber detalhes de seu funcionamento. A única interface que conhecemos são seus botões.

Classe
Liquidificador



Instância de um objeto de liquidificador: muito mais fácil de usar!

Encapsulamento

- Utiliza-se de modificadores de acesso, tais como:
  - **Public**: Acessível a partir de qualquer outra classe, independente do package.
  - **Protected**: Acessível a partir de qualquer classe derivada, no mesmo package.
  - **Private**: Acessível apenas dentro da própria classe.
  - **Static**: Membro compartilhado com todas as instâncias da classe.
  - **Final**: Constante da classe.
  - **Friendly**: Acessível a partir de qualquer outra classe do mesmo package (também chamado de “default” ou “package-private”).

Encapsulamento - Atributo

- Os atributos são características (variáveis) definidas em uma classe e podem ser classificados em dois tipos:
  - Atributos de Instância: são propriedades que pertencem a cada objeto individualmente e armazenam o estado específico de cada instância.
  - Atributos de Classe: são propriedades associadas à classe em si, cujo valor/estado é compartilhado por todas as instâncias dessa classe.

```
class CartaDeBaralho{
        public char Símbolo;
        public image Naipe;
        public static ListaDeSímbolos[] = {0,1,2,3,4,5,6,7,8,9,J,Q,K,A};
        public static ListaDeNaipes[] = { ♥ , ♠ , ♦ , ♣ };
}
```

Atributos de Instância

Atributos de Classe

Encapsulamento - Método

- Os métodos são rotinas executadas por um objeto ao receber uma mensagem, determinando seu comportamento. Eles são análogos às funções ou procedimentos da programação estruturada, mas estão associados a um contexto específico de um objeto ou classe.
  - É uma rotina que é executada por um objeto ao receber uma mensagem. De certo modo, é a descrição da forma como o objeto realiza uma tarefa.
  - Os métodos determinam o comportamento dos objetos de uma classe e são análogos às funções ou procedimentos da programação estruturada.
  - Assim como os atributos, os métodos podem ser encapsulados por meio de modificadores de acesso, controlando como eles podem ser chamados dentro e fora da classe.

Encapsulamento de atributos

Exemplo:

```java
package aularevisao;
class Data{
    private int dia, mes, ano;

    public int getDia() { return dia; }
    public void setDia(int dia) {
        if (dia > 0 && dia < 32) this.dia = dia;
    }
    public int getMes() { return mes; }
    public void setMes(int mes) {
        if (mes > 0 && mes < 13) this.mes = mes;
    }
    public int getAno() { return ano; }
    public void setAno(int ano) { this.ano = ano; }

    String retornarInfo(){
        return String.format("Data: %d/%d/%d", dia, mes, ano);
    }
}

public class Principal {
    public static void main(String[] args) {
        Data data = new Data();
        data.setDia(31); data.setMes(12);  data.setAno(2025);
        System.out.println(data.retornarInfo());
    }
}
```

Saída - AulaRevisao (run)

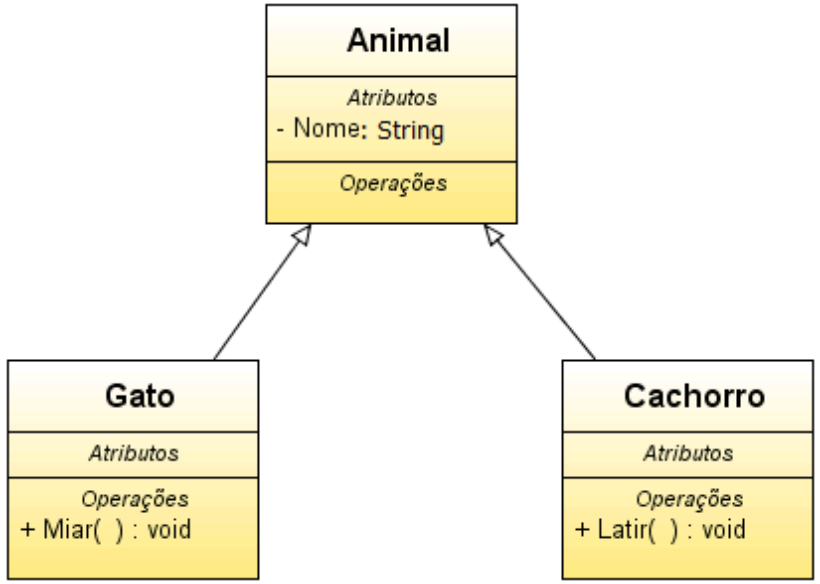
```
run:
Data: 31/12/2025
```

Herança - Definição

- Conhecida também como Generalização.
- Definição de uma classe a partir de outra, herdando os seus membros (não-privados).
- A classe herdeira é chamada subclasse.
- A classe herdada é chamada superclasse.
- Pode ser utilizado em cascata, criando hierarquias com várias gerações de classes.
- Permite que as classes compartilhem atributos e métodos baseados em um **relacionamento**.

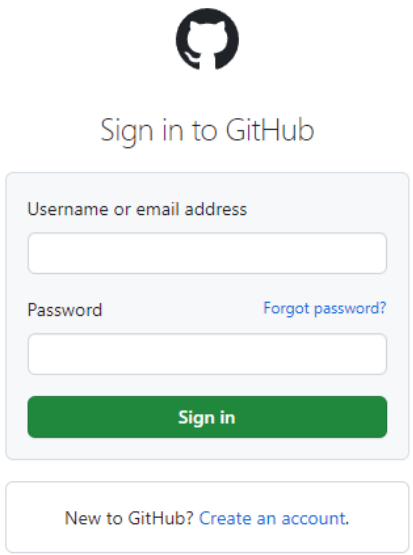
Subclasse e superclasse

- Superclasse é a classe herdada. Ou seja, a classe Pai.
- Subclasse é a classe herdeira. Ou seja, a classe Filha.
- No exemplo, Animal é a superclasse.
- No exemplo, Cachorro e Gato são subclasses.



Antes de prosseguir com código...

- Recomenda-se fortemente o uso e a familiarização com ferramentas de controle de versão, colaboração, integração etc., tais como o Git (e GitHub), pois é algo praticamente obrigatório no mercado de trabalho.
- Por isso, mas não somente, é importante aprender a utilizá-la e criar um portifólio com todos os seus códigos e projetos que estiver estudando ou desenvolvendo.
- Para auxiliar, segue um curso simples e gratuito: https://www.udemy.com/course/git-e-github-para-iniciantes/



Herança
Exemplo:

```java
abstract class Pessoa {

    protected String nome;
    private String email;

    public String getNome() { return nome; }

    public String getEmail() { return email; }

    public Pessoa(String nome, String email) {
        this.nome = nome;
        this.email = email;
    }
}
```

```java
public class Aluno extends Pessoa {

    private int ra;

    public Aluno(String nome, String email, int RA) {
        super(nome, email);
        this.ra = RA;
    }
}
```

```java
public class Professor extends Pessoa {

    public Professor(String nome, String email) {
        super(nome, email);
    }
}
```

Obrigado!